

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Modul 1

Pengantar Mikrokontroler

Lu'bah Al 'Aini - 5024241082

2026

1 Langkah-Langkah Percobaan

Seluruh percobaan dikerjakan dalam satu project PlatformIO bernama `blink` di Visual Studio Code (sistem operasi Linux). Pada setiap percobaan hanya isi `platformio.ini` dan berkas program pada folder `src/` yang diganti, sesuai petunjuk Bagian D modul. Perangkat keras dipasang pada papan praktikum Laboratorium B401 yang sudah menyediakan soket ESP32, soket STM32 Blackpill, breadboard, terminal sekrup bernomor, dan modul pushbutton.

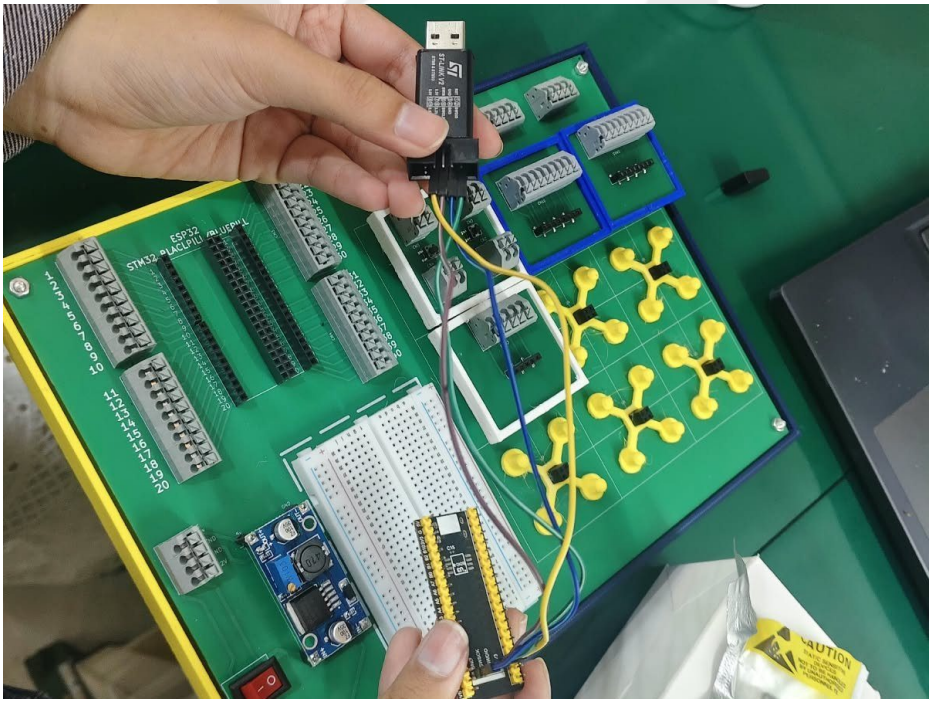
1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Isi `platformio.ini` diganti dengan konfigurasi Blackpill berikut, lalu ditunggu hingga PlatformIO selesai melakukan *refresh* konfigurasi.

Kode 1: `platformio.ini` Percobaan 1

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

2. ST-Link V2 dihubungkan ke Blackpill menggunakan empat kabel jumper *female-female* dengan pasangan pin SWDIO–SWDIO, SWCLK–SWCLK, GND–GND, dan 3.3V–3V3, kemudian ST-Link dicolokkan ke port USB laptop (Gambar 1).



Gambar 1: Pemasangan ST-Link V2 ke STM32 Blackpill melalui jalur SWD.

3. Kode Blink ditulis pada `src/main.cpp`, lalu tombol **Upload** ditekan. Pada upload pertama PlatformIO mengunduh terlebih dahulu *tool* `stm32flash`, `stm32duino`, dan `openocd` (Gambar 2(a)). Upload selesai dengan status `[SUCCESS]` dalam 67,58 detik dan log OpenOCD menampilkan `Programming Finished`, `Verified OK`, serta `Resetting Target` (Gambar 2(b)).

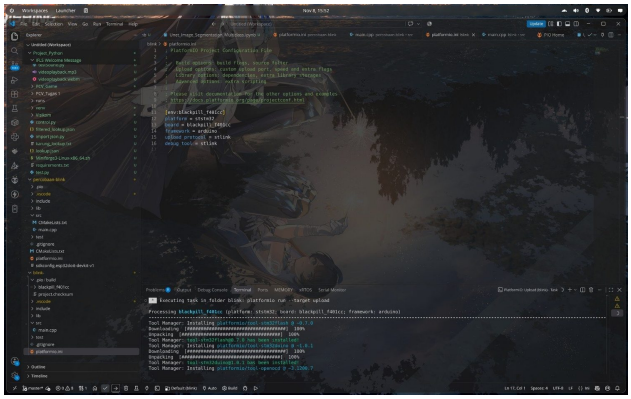
Kode 2: Program Blink STM32 (framework Arduino)

```
#include <Arduino.h>

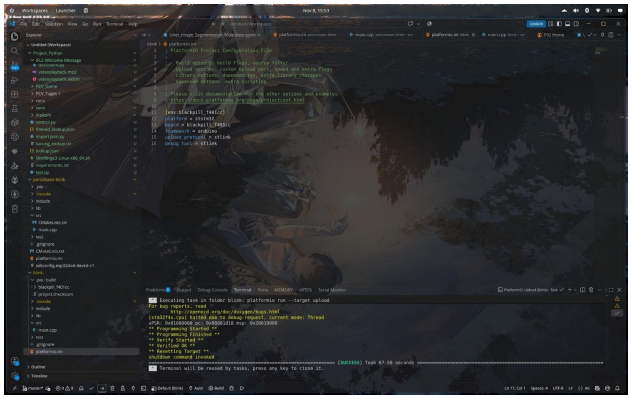
#define LED_PIN PC13

void setup() {
  pinMode(LED_PIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_PIN, LOW);
  delay(500);
  digitalWrite(LED_PIN, HIGH);
  delay(500);
}
```



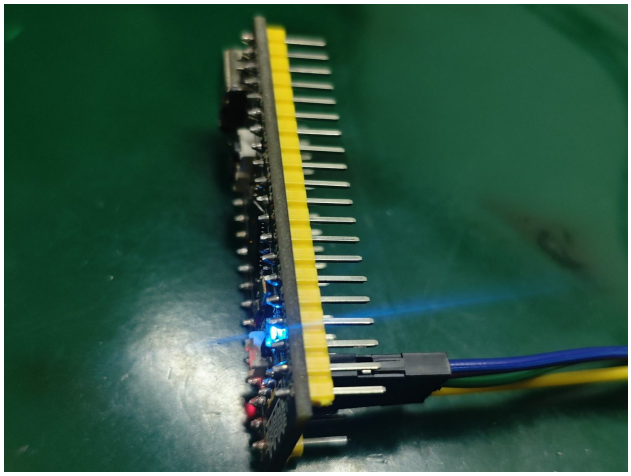
(a) Proses upload: instalasi *tool* ST-Link.



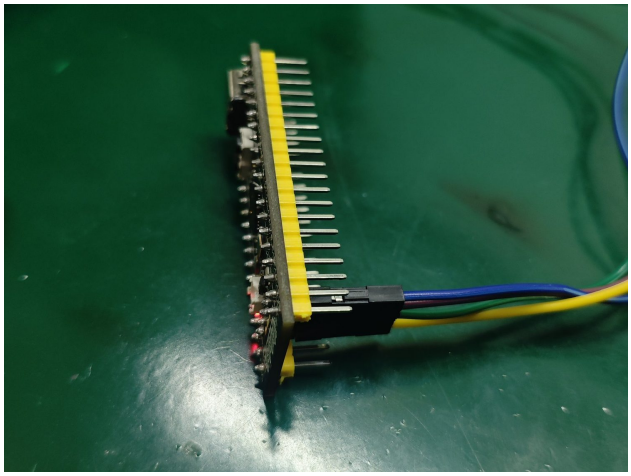
(b) Upload berhasil, Flash terverifikasi.

Gambar 2: Proses *build* dan upload program Blink ke Blackpill melalui ST-Link.

4. LED *onboard* PC13 diamati. LED menyala dan padam bergantian setiap 0,5 detik (Gambar 3).



(a) LED PC13 menyala (pin bernilai LOW).



(b) LED PC13 padam (pin bernilai HIGH).

Gambar 3: Hasil program Blink pada STM32 Blackpill.

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Isi `platformio.ini` diganti dengan konfigurasi ESP32 berframework ESP-IDF.

Kode 3: `platformio.ini` Percobaan 2

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
```

2. ESP32 DevKit dihubungkan ke laptop dengan kabel USB. Port terdeteksi sebagai `/dev/ttyUSB0` sehingga tidak diperlukan programmer tambahan.

3. Berkas `src/main.cpp` dihapus dan diganti dengan `src/main.c` berisi kode di bawah, lalu dilakukan upload. `esptool` menulis 147.568 byte (83.616 byte setelah kompresi) ke alamat `0x00010000`, memverifikasi *hash*, lalu me-reset board secara otomatis melalui pin RTS. Upload selesai dalam 7,54 detik (Gambar 4).

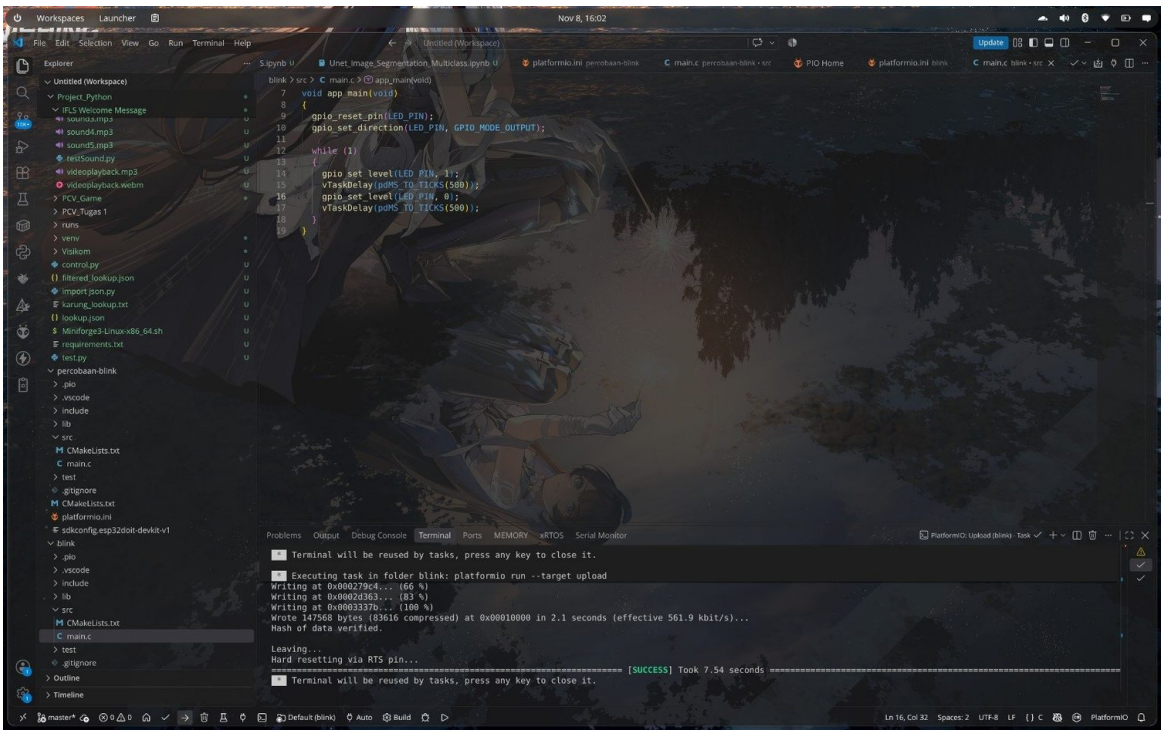
Kode 4: Program Blink ESP32 (framework ESP-IDF)

```c
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

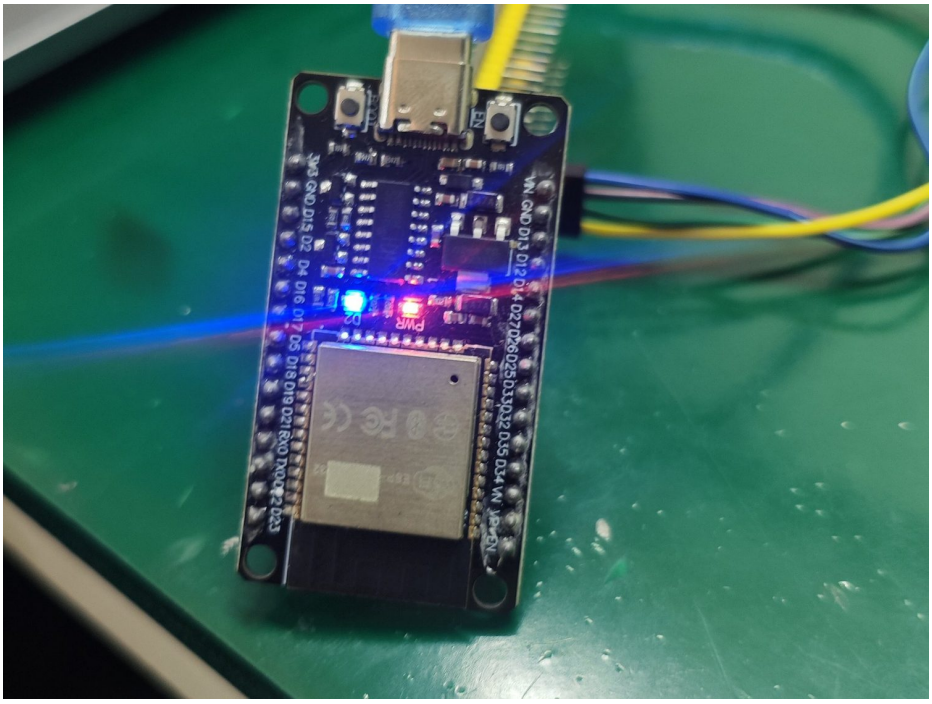
void app_main(void)
{
  gpio_reset_pin(LED_PIN);
  gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

  while (1)
  {
    gpio_set_level(LED_PIN, 1);
    vTaskDelay(pdMS_TO_TICKS(500));
    gpio_set_level(LED_PIN, 0);
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}
```



Gambar 4: Program Blink ESP-IDF pada `main.c` dan log upload `esptool` yang berhasil.

4. LED biru *onboard* (GPIO2) berkedip dengan periode 1 detik, sedangkan LED merah PWR tetap menyala sebagai indikator daya (Gambar 5). Perbedaan cara pemrograman ESP-IDF dan Arduino dianalisis pada Bagian 2.2.



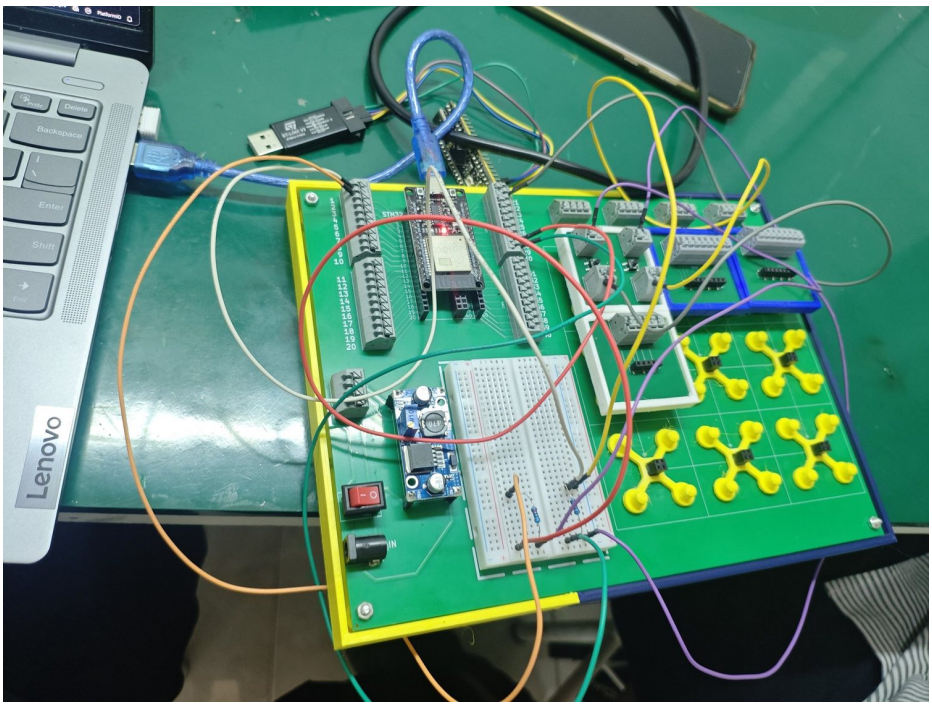
Gambar 5: LED *onboard* GPIO2 ESP32 menyala saat program Blink ESP-IDF berjalan.

1.3 Percobaan 3: Pull-up dan Pull-down, Eksternal vs Internal

1. `platformio.ini` dikembalikan ke framework Arduino (`platform = espressif32`, `board = esp32dev`, `framework = arduino`). Berkas `src/main.c` dihapus dan `src/main.cpp` dibuat

kembali.

2. Rangkaian Bagian A dipasang: Pushbutton 1 ke GPIO32 dengan resistor 10 kΩ ke 3,3 V (pull-up eksternal) dan kaki lain ke GND, Pushbutton 2 ke GPIO33 dengan resistor 10 kΩ ke GND (pull-down eksternal) dan kaki lain ke 3,3 V. Kedua resistor dipasang pada breadboard (Gambar 6).



Gambar 6: Rangkaian Bagian A: dua pushbutton dengan resistor pull-up dan pull-down eksternal pada breadboard.

3. Kode Program A diunggah. Pin dikonfigurasi sebagai `INPUT` biasa karena resistor sudah terpasang di luar. *Baud rate* Serial pada percobaan ini diatur 9600 dan Serial Monitor disesuaikan. Kedua tombol ditekan bergantian dan hasilnya dicatat (Gambar 7).

Kode 5: Kode Program A (pull-up dan pull-down eksternal)

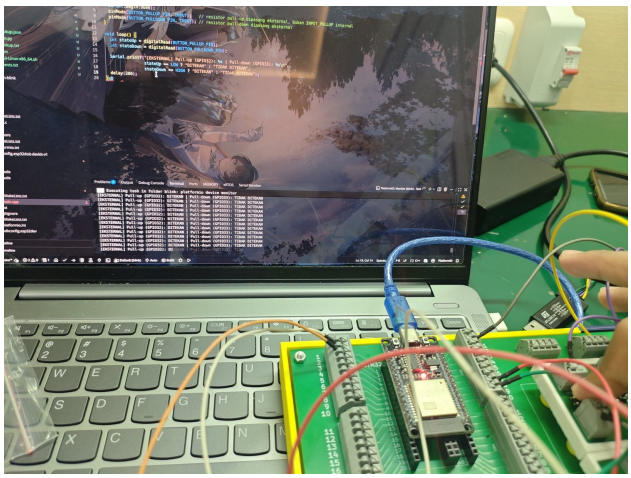
```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32
#define BUTTON_PULLDOWN_PIN 33

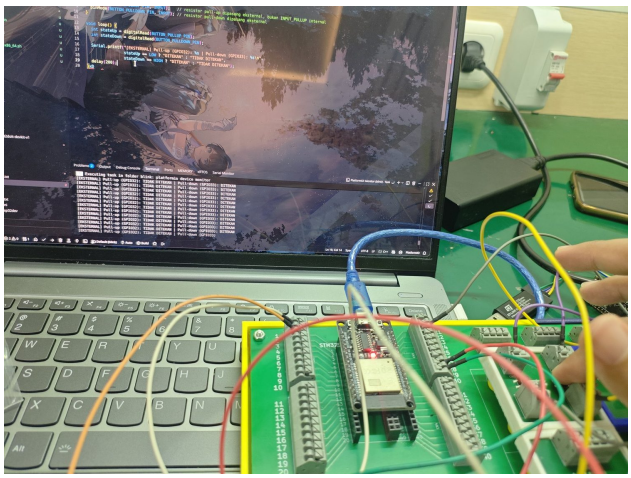
void setup() {
  Serial.begin(9600);
  pinMode(BUTTON_PULLUP_PIN, INPUT);
  pinMode(BUTTON_PULLDOWN_PIN, INPUT);
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[EKSTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```



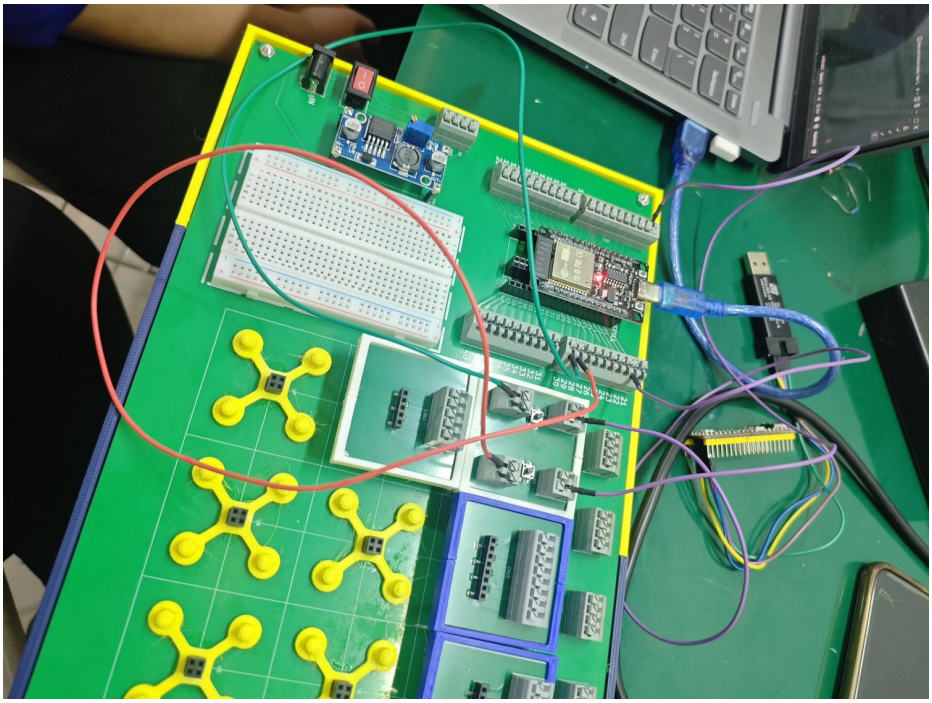
(a) Tombol pull-up (GPIO32) ditekan.



(b) Tombol pull-down (GPIO33) ditekan.

Gambar 7: Hasil pembacaan tombol dengan resistor eksternal.

4. Kedua resistor eksternal dilepas sesuai Skema Bagian B. Tombol tetap terpasang: Pushbutton 1 antara GPIO32 dan GND, Pushbutton 2 antara GPIO33 dan 3,3 V, dengan memperhatikan pasangan kaki tombol yang benar (Gambar 8).

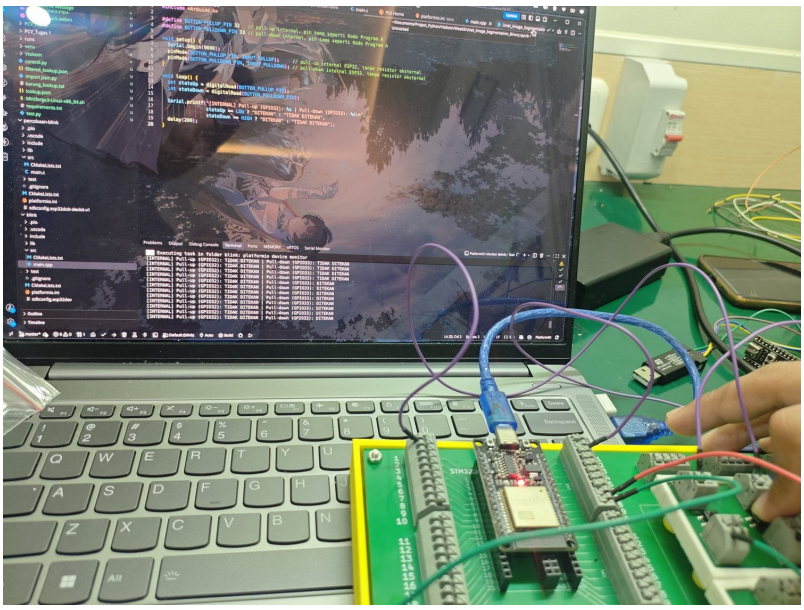


Gambar 8: Rangkaian Bagian B: tombol terhubung langsung ke GPIO32/33 tanpa resistor eksternal.

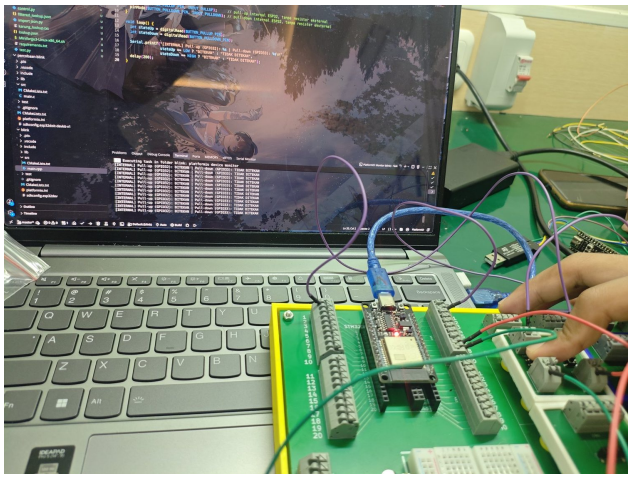
5. Kode Program B diunggah. Bedanya hanya pada mode pin, yaitu `INPUT_PULLUP` untuk GPIO32 dan `INPUT_PULLDOWN` untuk GPIO33. Tombol yang sama diuji kembali (Gambar 10).

Kode 6: Kode Program B (pull-up dan pull-down internal), bagian yang berbeda

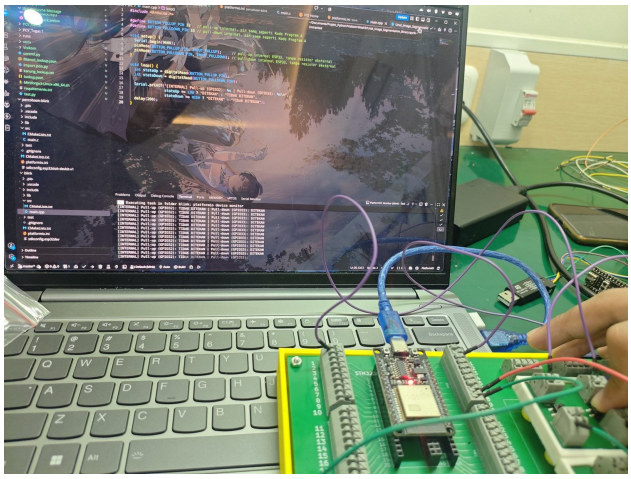
```
1 void setup() {
2   Serial.begin(9600);
3   pinMode(BUTTON_PULLUP_PIN, INPUT_PULLUP);
4   pinMode(BUTTON_PULLDOWN_PIN, INPUT_PULLDOWN);
5 }
```



Gambar 9: Transisi dari kondisi kedua tombol tidak ditekan menuju tombol pull-down (GPIO33) ditekan, pada mode internal.



(a) Tombol pull-up internal (GPIO32) ditekan.



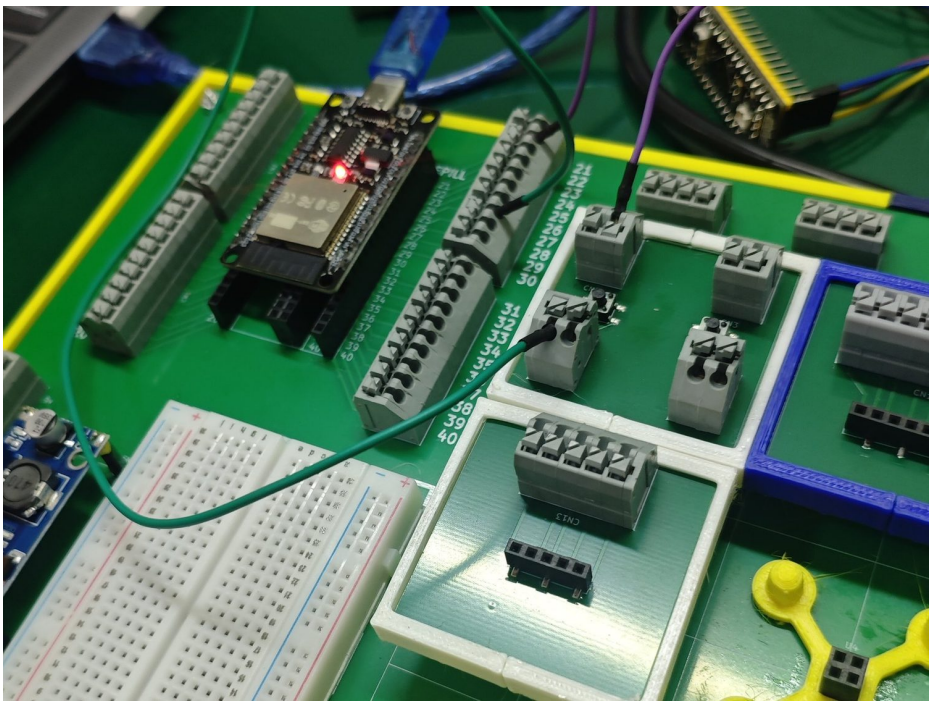
(b) Tombol pull-down internal (GPIO33) ditekan.

Gambar 10: Hasil pembacaan tombol dengan resistor internal ESP32.

6. Hasil A dan B dibandingkan. Keduanya identik, sehingga langkah 7 modul (pemeriksaan resistor yang belum dilepas) tidak diperlukan. Rekapitulasi ada pada Tabel 1.

1.4 Percobaan 4: Debouncing pada Input Tombol GPIO

1. Satu pushbutton dipasang antara GPIO25 dan GND tanpa resistor, memanfaatkan `INPUT_PULLUP` internal (Gambar 11). Konfigurasi `platformio.ini` sama dengan Percobaan 3.



Gambar 11: Rangkaian Percobaan 4: pushbutton pada GPIO25 (kabel hijau) dan GND (kabel ungu).

2. Kode pembanding `counterNoDebounce` dan `counterWithDebounce` dari modul ditulis pada `src/main.cpp` dengan `DEBOUNCE_DELAY 50`. *Baud rate* Serial diatur 9600.

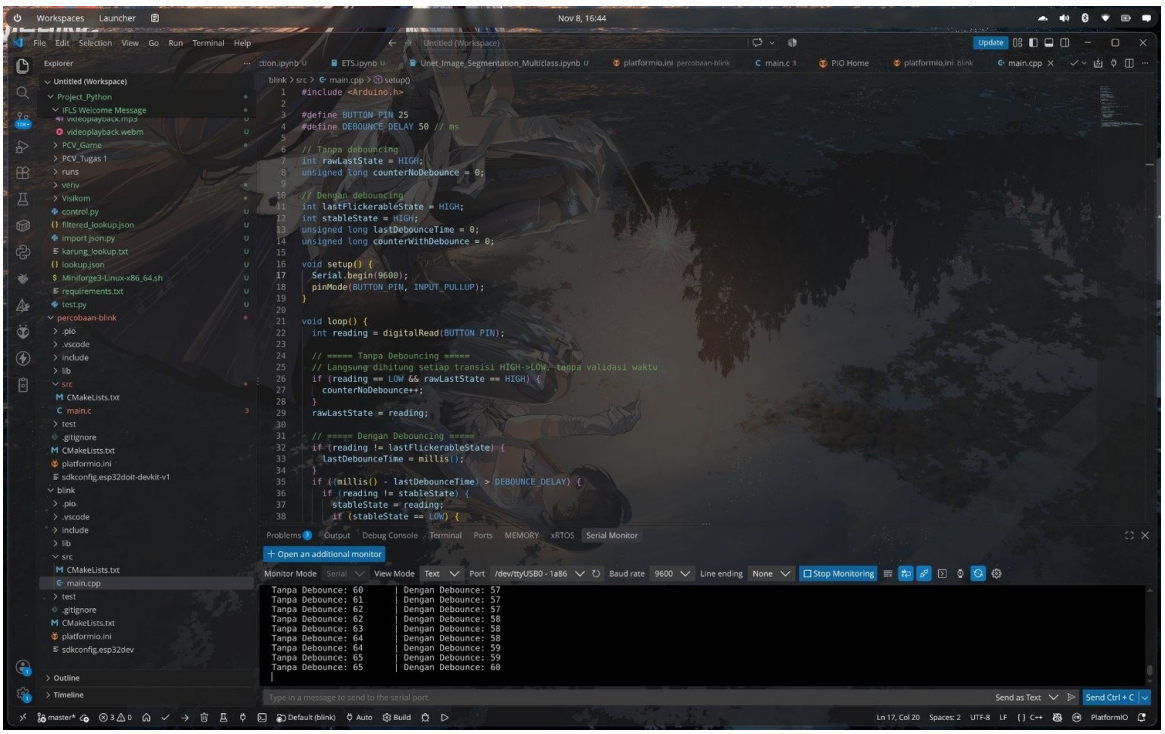
Kode 7: Inti logika penghitung tanpa dan dengan debouncing

```
int reading = digitalRead(BUTTON_PIN);

if (reading == LOW && rawLastState == HIGH) {
  counterNoDebounce++;
}
rawLastState = reading;

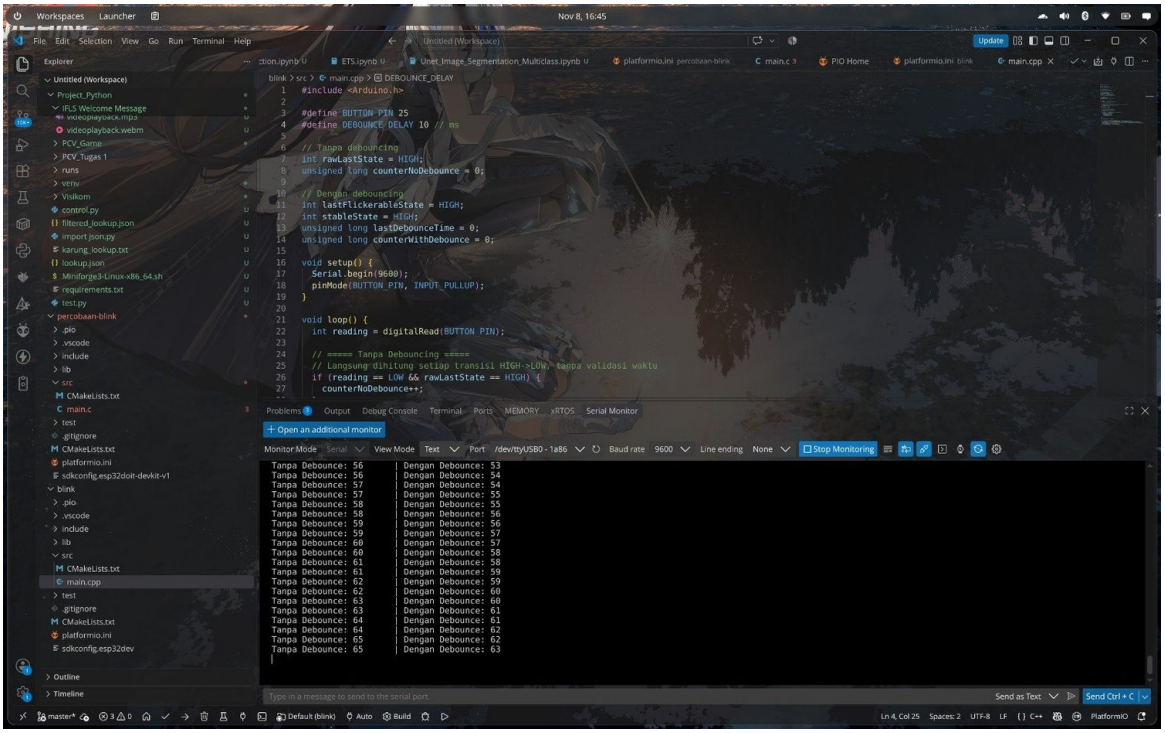
if (reading != lastFlickerableState) {
  lastDebounceTime = millis();
}
if ((millis() - lastDebounceTime) > DEBOUNCE_DELAY) {
  if (reading != stableState) {
    stableState = reading;
    if (stableState == LOW) {
      counterWithDebounce++;
    }
  }
}
lastFlickerableState = reading;
```

3. Tombol ditekan berulang kali dengan cara yang berbeda-beda (cepat dan lambat) sambil membandingkan kedua penghitung di Serial Monitor (Gambar 12).



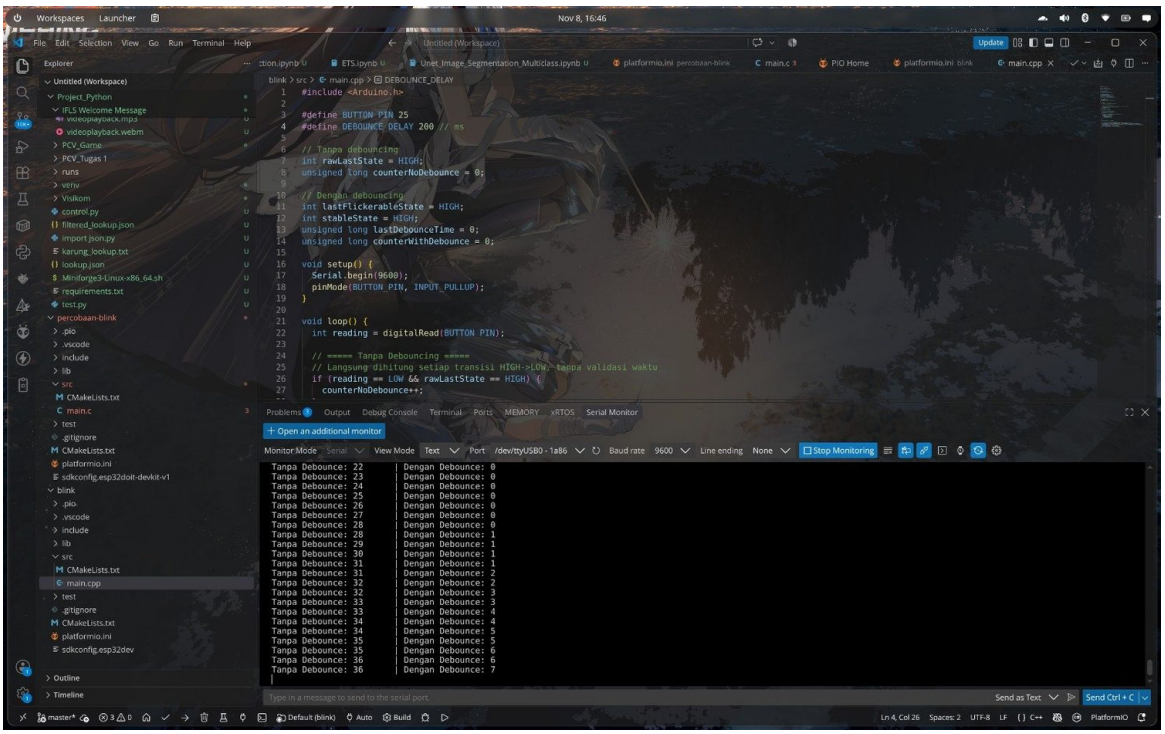
Gambar 12: Hasil `DEBOUNCE_DELAY` = 50 ms: akhir pengamatan 65 (tanpa debounce) dan 60 (dengan debounce).

4. `DEBOUNCE_DELAY` diubah menjadi 10 ms, diunggah ulang, lalu tombol ditekan kembali (Gambar 13).



Gambar 13: Hasil `DEBOUNCE_DELAY` = 10 ms: akhir pengamatan 65 (tanpa debounce) dan 63 (dengan debounce).

5. `DEBOUNCE_DELAY` diubah menjadi 200 ms, diunggah ulang, lalu tombol ditekan cepat berulang kali (Gambar 14).

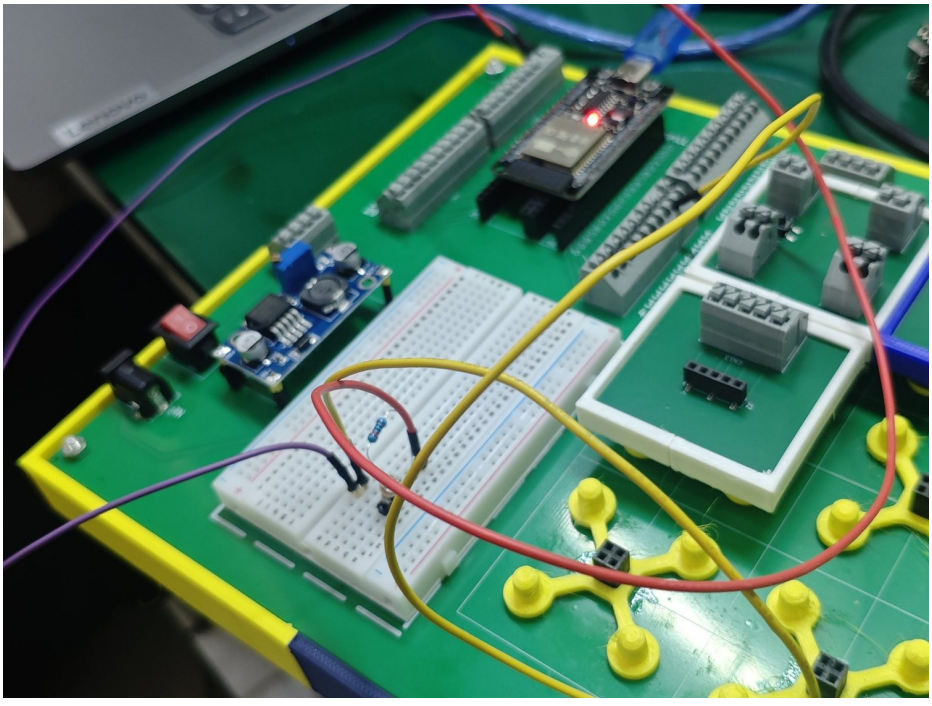


Gambar 14: Hasil `DEBOUNCE_DELAY` = 200 ms: akhir pengamatan 36 (tanpa debounce) dan hanya 7 (dengan debounce).

6. Ketiga hasil dibandingkan untuk melihat untung-rugi nilai *delay* (Tabel 2 dan Bagian 2.4).

1.5 Percobaan 5: Akses GPIO Analog dengan LDR

1. LDR dan resistor 10 kΩ dirangkai sebagai pembagi tegangan pada breadboard: satu kaki LDR ke 3,3 V, kaki lainnya ke GPIO34 sekaligus ke GND melalui resistor 10 kΩ (Gambar 15).



Gambar 15: Rangkaian pembagi tegangan LDR dan resistor 10 kΩ pada breadboard, titik tengah ke GPIO34.

2. Program pembacaan ADC ditulis. Nilai mentah `analogRead()` dikonversi ke tegangan dengan faktor $3{,}3/4095$.

Kode 8: Program pembacaan ADC dengan LDR

```
#include <Arduino.h>

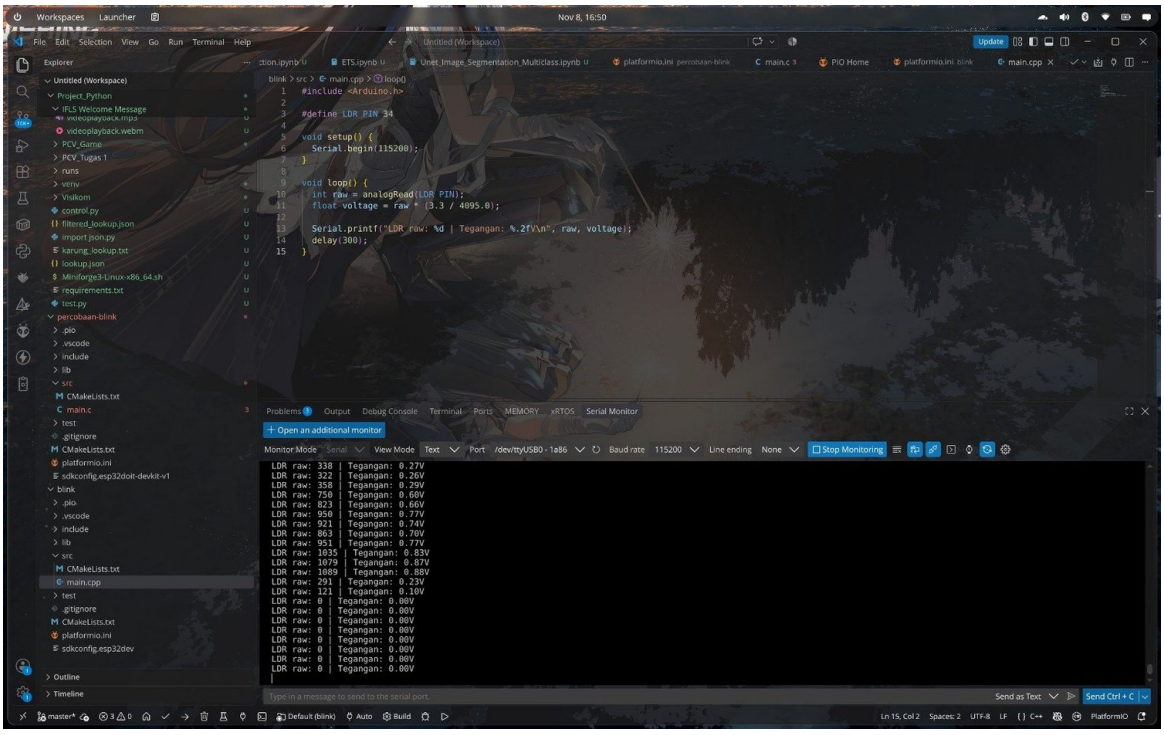
#define LDR_PIN 34

void setup() {
  Serial.begin(115200);
}

void loop() {
  int raw = analogRead(LDR_PIN);
  float voltage = raw * (3.3 / 4095.0);

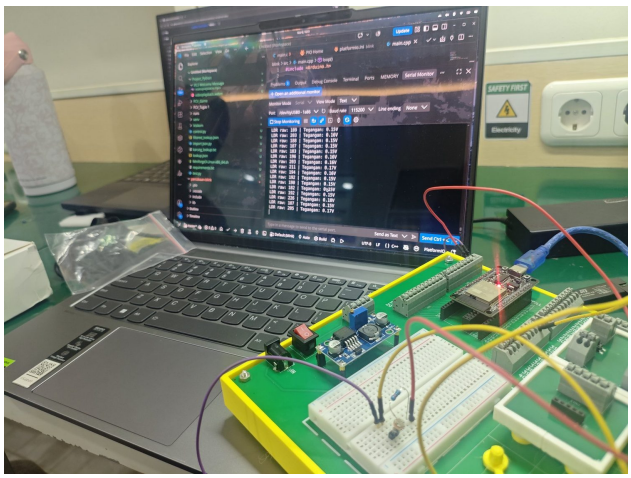
  Serial.printf("LDR raw: %d | Tegangan: %.2fV\n", raw, voltage);
  delay(300);
}
```

3. Program diunggah dan Serial Monitor dibuka pada 115200 baud. Gambar 16 memperlihatkan nilai yang berubah ketika LDR mendapat cahaya lebih terang (naik ke sekitar 1000) lalu ditutup (turun hingga 0).

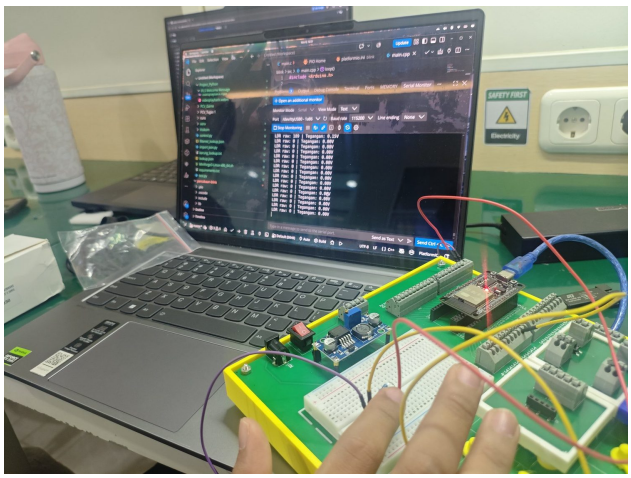


Gambar 16: Perubahan nilai ADC saat LDR disinari (raw 750–1089) lalu ditutup tangan (raw turun ke 0).

4. LDR diuji pada beberapa kondisi cahaya: cahaya ruangan, ditutup tangan, disinari senter, dan disinari sangat dekat (Gambar 17 dan 18).

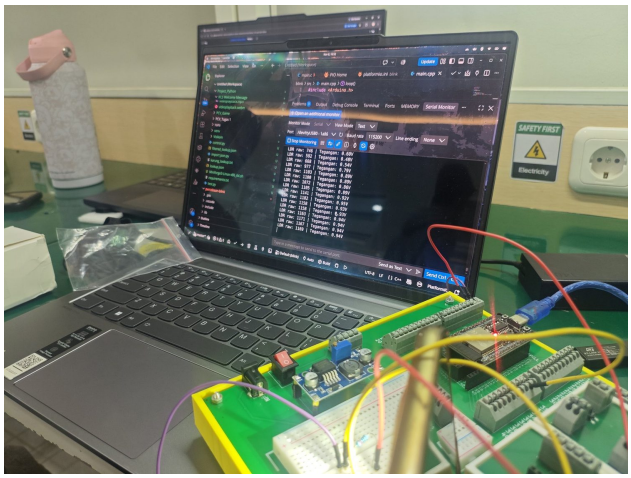


(a) Cahaya ruangan: raw 182–226 (0,15–0,18 V).

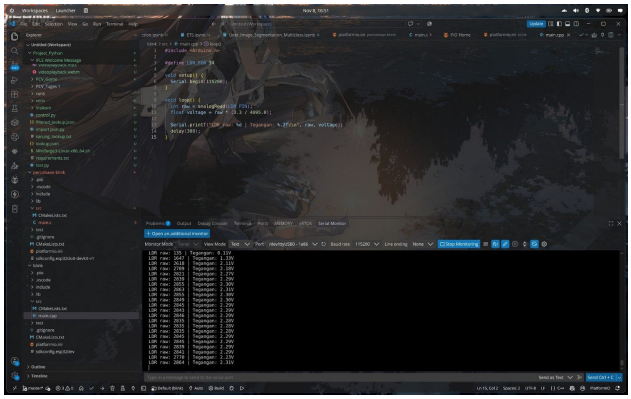


(b) LDR ditutup tangan: raw 0 (0,00 V).

Gambar 17: Pembacaan LDR pada kondisi cahaya ruangan dan gelap.

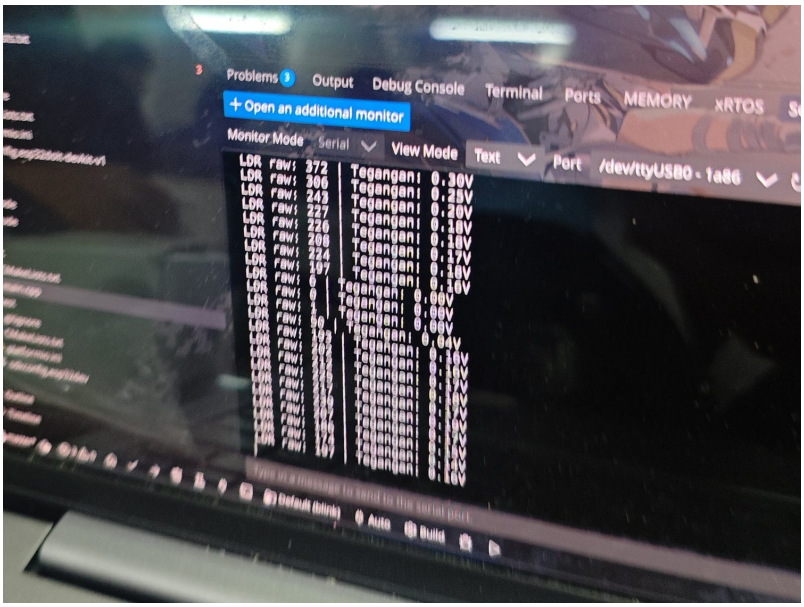


(a) Disinari senter: raw 1100–1182 (0,89–0,95 V).



(b) Disinari sangat dekat: raw 2770–2864 (2,23–2,31 V).

Gambar 18: Pembacaan LDR pada kondisi terang.



Gambar 19: Pengamatan nilai saat LDR berpindah dari cahaya ruangan ke kondisi tertutup dan kembali lagi.

5. Hasil dianalisis untuk menjelaskan mengapa sinyal analog dapat menampilkan banyak kondisi (Bagian 2.5).

2 Analisis Hasil Percobaan

2.1 Percobaan 1: Blink STM32 Blackpill

Program Blink berhasil di-*flash* ke Blackpill dan LED PC13 berkedip dengan periode 1 detik (0,5 s menyala, 0,5 s padam). Hasil ini sesuai teori bahwa LED *onboard* Blackpill aktif LOW: pada kode, perintah `digitalWrite(LED_PIN, LOW)` justru membuat LED menyala karena katoda LED terhubung ke pin, sedangkan anodanya ke 3,3 V melalui resistor.

Log upload menunjukkan bahwa PlatformIO memakai OpenOCD untuk berkomunikasi dengan ST-Link. OpenOCD menghentikan inti Cortex-M4 (`halted due to debug-request`), menulis Flash, memverifikasi isinya (`Verified OK`), lalu me-reset target. Seluruh proses ini dilakukan lewat jalur SWD tanpa bantuan program apa pun yang berjalan di chip, sehingga Blackpill tetap dapat diprogram meskipun Flash-nya kosong atau berisi program yang macet. Waktu upload pertama yang lama (67,58 s) disebabkan PlatformIO mengunduh paket `tool-stm32flash`, `tool-stm32duino`, dan `tool-openocd`. Upload berikutnya hanya memerlukan beberapa detik.

2.2 Percobaan 2: Blink ESP32 dengan ESP-IDF

LED GPIO2 berkedip dengan periode yang sama seperti Percobaan 1, tetapi aktif HIGH: LED menyala ketika pin diberi level 1 melalui `gpio_set_level()`. Proses upload jauh lebih sederhana: cukup satu kabel USB, karena chip USB-to-Serial pada ESP32 DevKit meneruskan data ke bootloader ROM ESP32 melalui UART, dan sinyal RTS/DTR dipakai untuk masuk ke mode *download* serta me-reset board secara otomatis (`Hard resetting via RTS pin`). Tidak diperlukan penekanan tombol BOOT.

Perbedaan memprogram dengan ESP-IDF dibandingkan Arduino yang teramati:

- **Struktur program.** ESP-IDF tidak memiliki `setup()` dan `loop()`. Program dimulai dari `app_main()` yang berjalan sebagai *task* FreeRTOS, sehingga perulangan harus ditulis sendiri dengan `while (1)`.
- **Bahasa dan berkas.** Program ESP-IDF ditulis dalam bahasa C pada `main.c` dan memerlukan *include* eksplisit (`FreeRTOS.h`, `task.h`, `driver/gpio.h`). Pada Arduino cukup `#include <Arduino.h>`.
- **Akses GPIO.** `gpio_reset_pin()` dan `gpio_set_direction()` menggantikan `pinMode()`, sedangkan `gpio_set_level()` menggantikan `digitalWrite()`. Nomor pin ditulis sebagai konstanta `GPIO_NUM_2`.
- **Penundaan.** `vTaskDelay(pdMS_TO_TICKS(500))` menyerahkan CPU ke *task* lain selama menunggu, berbeda dengan `delay()` yang secara konsep bersifat *blocking*.
- **Proses build.** Project ESP-IDF menghasilkan berkas tambahan seperti `CMakeLists.txt` dan `sdkconfig`, yang menandakan sistem build berbasis CMake dan konfigurasi *menuconfig*. Ukuran biner yang ditulis sekitar 144 KB.

Kode ESP-IDF memang lebih panjang untuk tugas yang sama, tetapi setiap langkahnya eksplisit dan dekat ke perangkat keras, sesuai perbandingan pada laporan sementara.

2.3 Percobaan 3: Pull-up dan Pull-down

Tabel 1: Hasil pembacaan tombol pada konfigurasi eksternal dan internal

Konfigurasi	Kondisi tombol	Eksternal (A)	Internal (B)
Pull-up (GPIO32)	Tidak ditekan Ditekan	HIGH (TIDAK DITEKAN) LOW (DITEKAN)	HIGH (TIDAK DITEKAN) LOW (DITEKAN)
Pull-down (GPIO33)	Tidak ditekan Ditekan	LOW (TIDAK DITEKAN) HIGH (DITEKAN)	LOW (TIDAK DITEKAN) HIGH (DITEKAN)

Hasil pada Tabel 1 sesuai teori. Pada konfigurasi pull-up, pin ditarik ke 3,3 V sehingga terbaca HIGH saat tombol dilepas, dan menjadi LOW ketika tombol menghubungkan pin ke GND. Pull-down bekerja sebaliknya. Pesan yang tampil selalu konsisten dan tidak pernah berubah acak saat tombol dilepas, yang menunjukkan pin tidak dalam kondisi *floating*.

Resistor eksternal dan internal memberikan hasil logika yang identik. Perbedaannya ada pada nilai resistansi: resistor pull internal ESP32 bernilai sekitar 45 kΩ, lebih besar dari 10 kΩ eksternal. Arus saat tombol ditekan menjadi lebih kecil (sekitar 73 μA dibanding 330 μA), tetapi pin juga lebih mudah terpengaruh derau jika kabelnya panjang. Untuk jarak kabel pendek seperti di papan praktikum, pull internal sudah cukup dan menghemat komponen. Keuntungan lain yang terlihat adalah rangkaian Bagian B jauh lebih ringkas (Gambar 8) dibanding Bagian A yang memerlukan breadboard (Gambar 6). Perlu diingat bahwa cara internal tidak berlaku untuk GPIO34–39 karena pin tersebut tidak memiliki resistor pull internal.

2.4 Percobaan 4: Debouncing

Tabel 2: Nilai akhir penghitung pada tiap nilai `DEBOUNCE_DELAY`

`DEBOUNCE_DELAY`	Tanpa debounce	Dengan debounce	Selisih
10 ms	65	63	2
50 ms	65	60	5
200 ms	36	7	29

Pada ketiga pengujian, `counterNoDebounce` selalu lebih besar dari `counterWithDebounce`. Kelebihan hitungan ini berasal dari *bouncing*: saat kontak logam tombol bersentuhan, sinyal bergetar HIGH–LOW beberapa kali sehingga setiap getaran dihitung sebagai penekanan baru oleh penghitung tanpa debounce. Bouncing tidak muncul pada setiap penekanan, sehingga selisihnya bertambah sedikit demi sedikit, sesuai catatan modul.

Pengaruh nilai `DEBOUNCE_DELAY` terlihat jelas:

- **10 ms.** Selisihnya paling kecil (2). Hal ini bukan berarti 10 ms paling baik. Jendela 10 ms terlalu pendek sehingga sebagian getaran yang berlangsung lebih lama dari 10 ms ikut lolos dan dihitung juga oleh penghitung ber-debounce, sehingga nilainya mendekati penghitung mentah.
- **50 ms.** Getaran tersaring dengan baik (5 hitungan palsu dibuang), sementara setiap penekanan normal tetap terhitung. Nilai ini merupakan kompromi yang paling seimbang.

- **200 ms.** Ketika tombol ditekan cepat berulang kali, penghitung ber-debounce hanya bertambah 7 dari 36 transisi. Selang antartekanan lebih pendek dari 200 ms sehingga sinyal tidak pernah dianggap stabil dan penekanan yang sah pun ikut terbuang. Pada awal pengamatan, nilai debounce bahkan tertahan di 0 ketika penghitung mentah sudah mencapai 22.

Hasil ini membuktikan untung-rugi yang dijelaskan modul: semakin besar nilai *delay*, semakin bersih dari getaran, tetapi tombol semakin lambat merespons dan penekanan cepat bisa hilang. Perlu dicatat bahwa urutan penekanan pada tiap pengujian dilakukan secara manual dan tidak identik, sehingga angka pada Tabel 2 menunjukkan kecenderungan, bukan pengukuran yang sepenuhnya terkontrol.

2.5 Percobaan 5: ADC dan LDR

Dengan menyusun ulang Persamaan pembagi tegangan pada laporan sementara, resistansi LDR dapat diperkirakan dari tegangan terukur:

$$R_{LDR} = R_1 \left(\frac{V_{in}}{V_o} - 1 \right), \qquad R_1 = 10\,\mathrm{k\Omega},\ V_{in} = 3{,}3\,\mathrm{V} \tag{1}$$

Tabel 3: Pembacaan ADC dan estimasi resistansi LDR pada berbagai kondisi cahaya

Kondisi	**Nilai raw**	**Tegangan (V)**	**Estimasi R_{LDR}**
Ditutup tangan	0	0,00	$\geq 300\,\mathrm{k\Omega}$
Cahaya ruangan	182–226	0,15–0,18	173–210 kΩ
Disinari senter	1100–1182	0,89–0,95	25–27 kΩ
Disinari sangat dekat	2770–2864	2,23–2,31	4,3–4,8 kΩ

Nilai ADC naik ketika cahaya bertambah dan turun ketika LDR ditutup, sesuai teori: resistansi LDR mengecil saat terang sehingga bagian tegangan pada R_1 (yang dibaca ADC) membesar. Dari keadaan gelap hingga sangat terang, resistansi LDR berubah hampir dua orde besaran (dari ratusan kΩ hingga beberapa kΩ).

Berbeda dengan Percobaan 3 dan 4 yang hanya mengenal HIGH dan LOW, pembacaan ADC menghasilkan banyak nilai di antaranya, misalnya 189, 750, 1089, dan 2864. ADC 12-bit ESP32 membagi rentang tegangan menjadi 4096 tingkat dengan langkah sekitar 0,8 mV, sehingga perubahan cahaya yang kecil pun terlihat sebagai perubahan angka, bahkan saat kondisi dianggap tetap (fluktuasi raw 182–226 pada cahaya ruangan akibat derau dan perubahan cahaya sekitar).

Beberapa hal yang perlu dicermati:

- **Pembacaan 0 saat ditutup.** Tegangan sebenarnya belum tentu tepat 0 V. ADC ESP32 kurang linear di dekat 0 V dan tegangan di bawah sekitar 0,1 V cenderung terbaca 0. Oleh karena itu batas resistansi pada baris pertama Tabel 3 hanya berupa perkiraan batas bawah.
- **Rentang kerja kurang optimal.** Pada cahaya ruangan tegangan hanya sekitar 0,16 V, sangat dekat dengan zona tidak linear tersebut. Agar resolusi pada kondisi normal lebih baik, nilai R_1 dapat diperbesar (misalnya 100 kΩ) sehingga tegangan cahaya ruangan berada di tengah rentang ADC.
- **Konversi tegangan bersifat pendekatan.** Rumus $raw \times 3{,}3/4095$ mengasumsikan ADC linear sempurna dengan referensi tepat 3,3 V. Untuk hasil yang lebih akurat, dapat digunakan `analogReadMilliVolts()` yang memakai data kalibrasi eFuse, serta dirata-ratakan beberapa sampel.

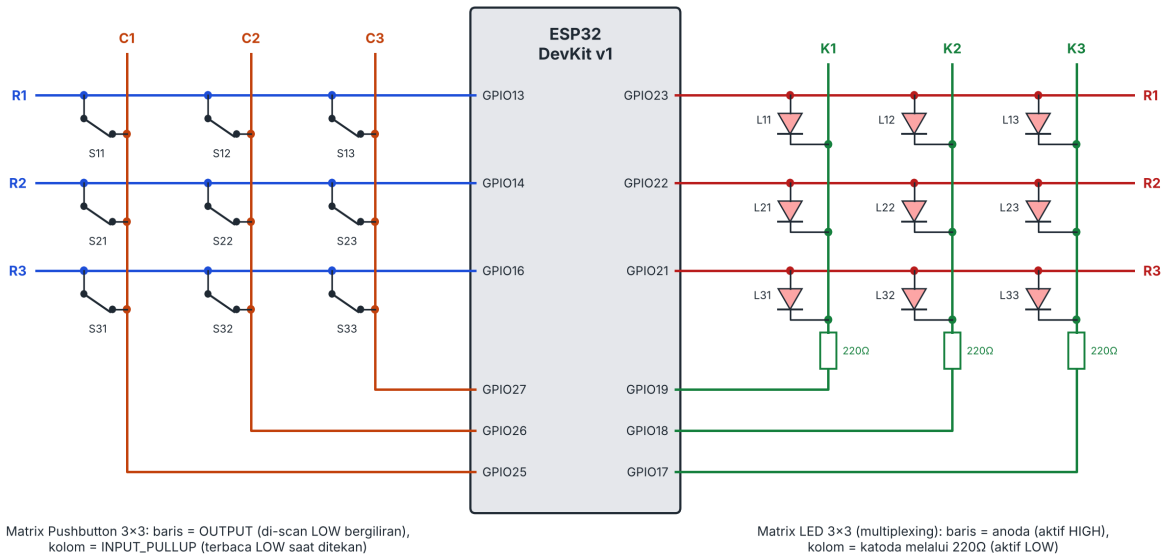
3 Hasil Tugas Modul

3.1 Rancangan Matrix Pushbutton 3x3 dan Matrix LED 3x3

Tugas dikerjakan pada simulator Wokwi menggunakan board ESP32 (Bagian 3.3). Kedua matriks dirangkai secara *multiplexing* agar sembilan tombol dan sembilan LED cukup dikendalikan oleh 12 GPIO, bukan 18. Pemetaan pin ditunjukkan pada Tabel 4 dan skematiknya pada Gambar 20.

Tabel 4: Pemetaan pin ESP32 pada tugas modul

Bagian	Fungsi	GPIO	Mode
Pushbutton	Baris R1, R2, R3	13, 14, 16	`OUTPUT`, aktif LOW saat di-scan
Pushbutton	Kolom C1, C2, C3	25, 26, 27	`INPUT_PULLUP`
LED	Baris (anoda) R1, R2, R3	23, 22, 21	`OUTPUT`, aktif HIGH
LED	Kolom (katoda) K1, K2, K3	19, 18, 17	`OUTPUT`, aktif LOW via 220 Ω



Gambar 20: Skematik rangkaian Matrix Pushbutton 3x3 dan Matrix LED 3x3 pada ESP32.

Pin yang dipilih menghindari pin *strapping* yang sensitif saat *boot* (GPIO0, 2, 12, 15), pin Flash (GPIO6–11), serta pin *input-only* GPIO34–39 yang tidak memiliki pull-up internal. Resistor 220 Ω cukup dipasang satu per kolom karena pada satu waktu hanya satu baris LED yang aktif, sehingga setiap resistor hanya dilalui arus satu LED.

3.2 Cara Kerja Program

1. **Scanning tombol.** Semua baris tombol dijaga HIGH. Secara bergiliran satu baris ditarik LOW, lalu ketiga kolom dibaca. Karena kolom memakai `INPUT_PULLUP`, kolom yang terbaca LOW menandakan tombol pada perpotongan baris dan kolom tersebut sedang ditekan.
2. **Debouncing per tombol.** Setiap tombol memiliki variabel sendiri, yaitu `lastReading`, `stableState`, dan `lastChange`, dengan algoritma yang sama seperti Percobaan 4. Status tombol baru dianggap berubah jika pembacaannya stabil lebih dari `DEBOUNCE_DELAY` = 50 ms (nilai terbaik dari Percobaan 4).

3. **Aksi LED.** Setiap kali tombol (r, c) terdeteksi stabil ditekan (transisi ke LOW), status LED (r, c) di-*toggle* dan kejadian tersebut dicetak ke Serial Monitor. Mode *toggle* dipilih karena bouncing yang tidak tersaring akan langsung terlihat sebagai LED yang menyala lalu padam lagi, sehingga efek debouncing mudah dibuktikan.

4. **Multiplexing LED.** Setiap 2 ms satu baris LED diaktifkan (anoda HIGH) dan katoda kolom diatur LOW untuk LED yang harus menyala. Satu siklus tiga baris berlangsung 6 ms ($\approx$167 Hz), jauh di atas batas kedipan yang dapat dilihat mata, sehingga kesembilan LED tampak menyala bersamaan. Pewaktuan memakai `micros()` sehingga tidak ada `delay()` yang menghambat pembacaan tombol.

Kode 9: Source code tugas modul (`src/main.cpp`)

```
#include <Arduino.h>

#define DEBOUNCE_DELAY 50
#define LED_ROW_PERIOD_US 2000

const uint8_t KP_ROW[3] = {13, 14, 16};
const uint8_t KP_COL[3] = {25, 26, 27};
const uint8_t LED_ROW[3] = {23, 22, 21};
const uint8_t LED_COL[3] = {19, 18, 17};

bool ledState[3][3] = {};
int lastReading[3][3];
int stableState[3][3];
unsigned long lastChange[3][3] = {};

uint8_t activeLedRow = 0;
unsigned long lastLedSwitch = 0;

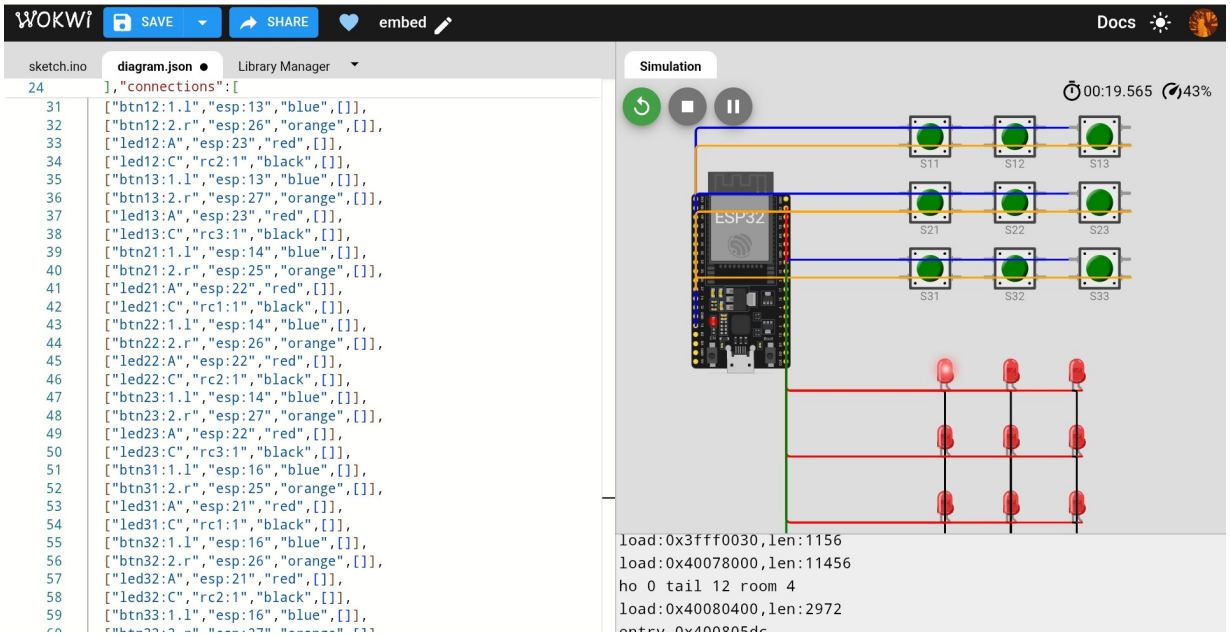
void scanKeypad() {
  for (uint8_t r = 0; r < 3; r++) {
    digitalWrite(KP_ROW[r], LOW);
    delayMicroseconds(5);
    for (uint8_t c = 0; c < 3; c++) {
      int reading = digitalRead(KP_COL[c]);
      if (reading != lastReading[r][c]) {
        lastChange[r][c] = millis();
        lastReading[r][c] = reading;
      }
      if ((millis() - lastChange[r][c]) > DEBOUNCE_DELAY && reading != stableState[r
    ][c]) {
        stableState[r][c] = reading;
        if (stableState[r][c] == LOW) {
          ledState[r][c] = !ledState[r][c];
          Serial.printf("Tombol (%d,%d) -> LED (%d,%d) %s\n", r + 1, c + 1, r + 1, c
    + 1,
                        ledState[r][c] ? "ON" : "OFF");
        }
      }
    }
    digitalWrite(KP_ROW[r], HIGH);
  }
}
```

```
41
42 void refreshLedMatrix() {
43   if (micros() - lastLedSwitch < LED_ROW_PERIOD_US) return;
44   lastLedSwitch = micros();
45
46   digitalWrite(LED_ROW[activeLedRow], LOW);
47   activeLedRow = (activeLedRow + 1) % 3;
48   for (uint8_t c = 0; c < 3; c++) {
49     digitalWrite(LED_COL[c], ledState[activeLedRow][c] ? LOW : HIGH);
50   }
51   digitalWrite(LED_ROW[activeLedRow], HIGH);
52 }
53
54 void setup() {
55   Serial.begin(115200);
56   for (uint8_t i = 0; i < 3; i++) {
57     pinMode(KP_ROW[i], OUTPUT);
58     digitalWrite(KP_ROW[i], HIGH);
59     pinMode(KP_COL[i], INPUT_PULLUP);
60     pinMode(LED_ROW[i], OUTPUT);
61     digitalWrite(LED_ROW[i], LOW);
62     pinMode(LED_COL[i], OUTPUT);
63     digitalWrite(LED_COL[i], HIGH);
64   }
65   for (uint8_t r = 0; r < 3; r++)
66     for (uint8_t c = 0; c < 3; c++) {
67       lastReading[r][c] = HIGH;
68       stableState[r][c] = HIGH;
69     }
70   Serial.println("Matrix Pushbutton 3x3 -> Matrix LED 3x3 siap");
71 }
72
73 void loop() {
74   scanKeypad();
75   refreshLedMatrix();
76 }
```

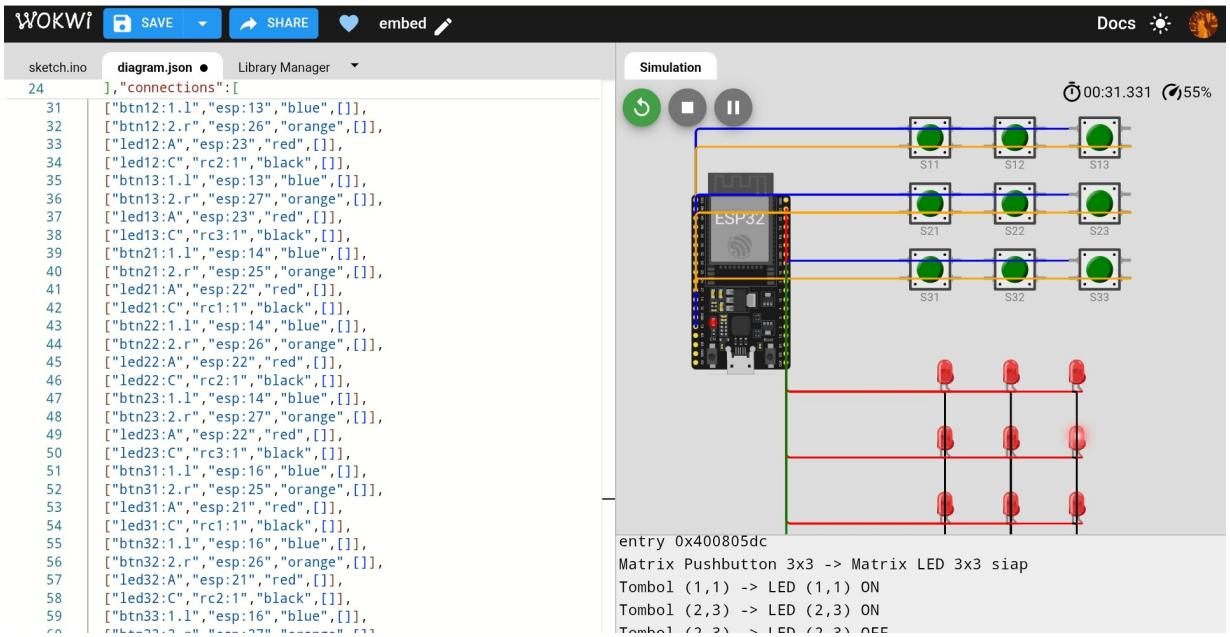
3.3 Hasil Simulasi

Simulasi dijalankan pada Wokwi (`https://wokwi.com/projects/480553227760372737`) dengan board ESP32 DevKitC, yang nomor GPIO-nya sama dengan ESP32 DevKit pada praktikum. Rangkaian terdiri atas sembilan pushbutton S11–S33, sembilan LED, dan tiga resistor 220 Ω sesuai Tabel 4. Program berhasil dikompilasi dan Serial Monitor menampilkan pesan `Matrix Pushbutton 3x3 -> Matrix LED 3x3 siap`.

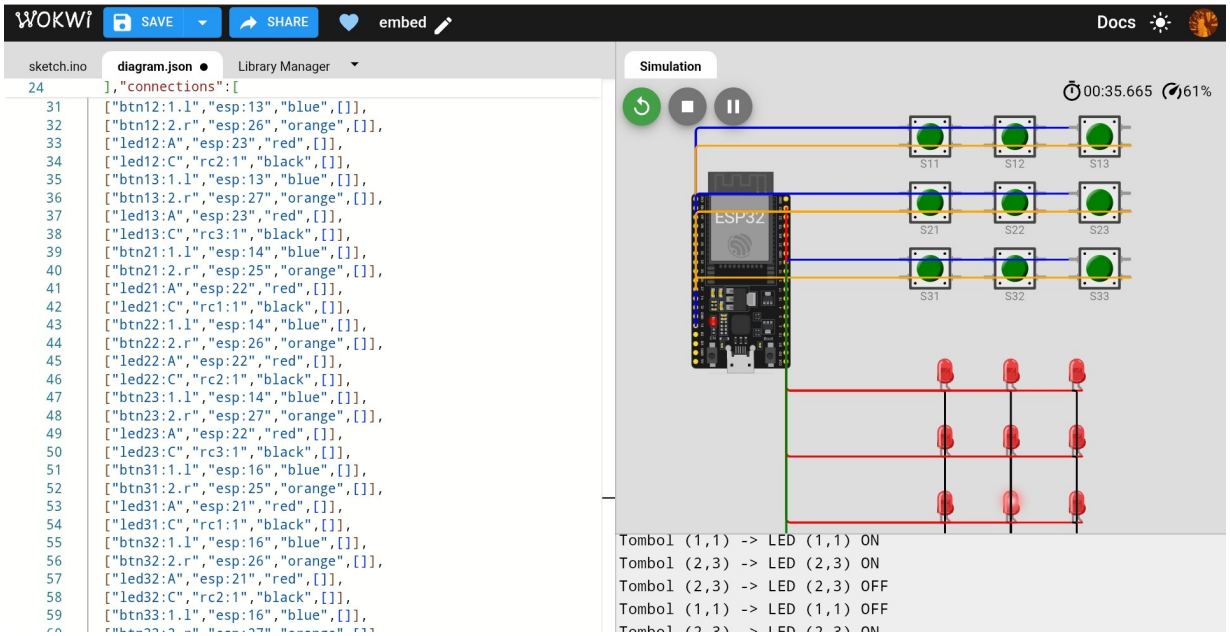
Tombol S11, S23, dan S32 ditekan bergantian. Setiap penekanan menyalakan LED pada posisi baris dan kolom yang sama, dan penekanan berikutnya pada tombol yang sama memadamkannya kembali (*toggle*), seperti terlihat pada Gambar 21–23.



Gambar 21: Simulasi Wokwi: tombol S11 ditekan, LED baris 1 kolom 1 menyala.



Gambar 22: Simulasi Wokwi: tombol S23 ditekan, LED baris 2 kolom 3 menyala. Serial Monitor mencatat `Tombol (1,1) -> LED (1,1) ON` dan `Tombol (2,3) -> LED (2,3) ON`.



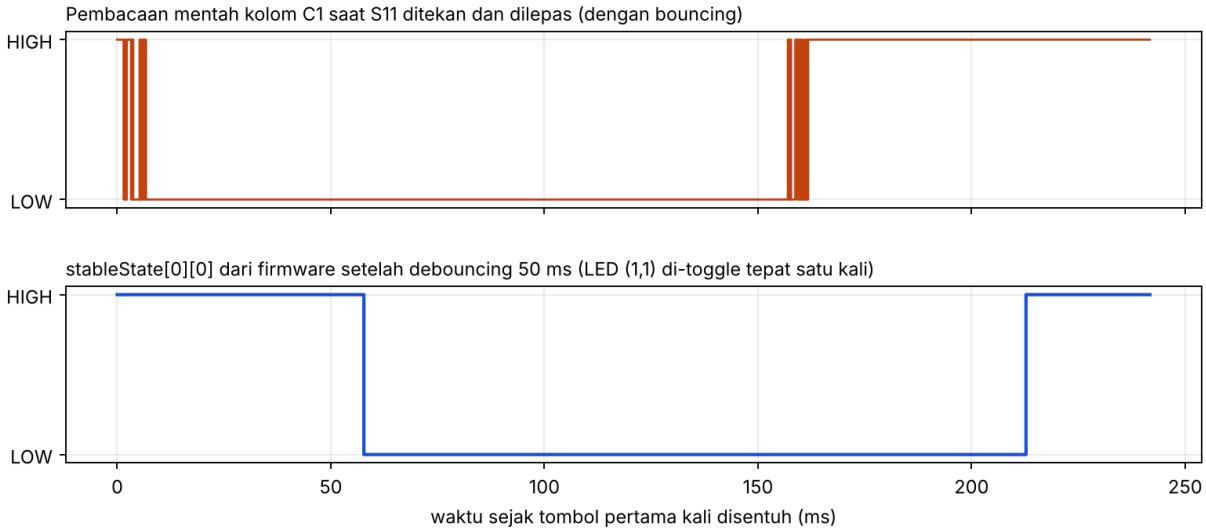
Gambar 23: Simulasi Wokwi: tombol S32 ditekan, LED baris 3 kolom 2 menyala. Log memperlihatkan LED (1,1) dan (2,3) dapat dipadamkan dan dinyalakan kembali.

Potongan keluaran Serial Monitor selama pengujian:

```
Matrix Pushbutton 3x3 -> Matrix LED 3x3 siap
Tombol (1,1) -> LED (1,1) ON
Tombol (2,3) -> LED (2,3) ON
Tombol (2,3) -> LED (2,3) OFF
Tombol (1,1) -> LED (1,1) OFF
Tombol (2,3) -> LED (2,3) ON
```

Pada tangkapan layar hanya terlihat LED dari satu baris pada satu waktu. Hal ini merupakan akibat multiplexing: program hanya mengaktifkan satu baris LED setiap 2 ms, dan tangkapan layar merekam satu saat saja, terlebih simulasi berjalan pada 43–61% kecepatan nyata. Pada rangkaian sebenarnya pergantian baris sekitar 167 Hz terlalu cepat untuk diikuti mata sehingga semua LED yang berstatus ON tampak menyala bersamaan.

Untuk memeriksa debouncing secara lebih terkontrol, source code yang sama juga diuji di luar Wokwi dengan model masukan yang sengaja diberi *bouncing* acak selama 7 ms saat tombol ditekan dan 5 ms saat dilepas. Gambar 24 memperlihatkan hasilnya untuk satu kali penekanan S11: sinyal mentah kolom C1 berganti HIGH–LOW sebanyak 14 kali, tetapi `stableState` di dalam program hanya berubah satu kali ke LOW (sekitar 50 ms setelah getaran terakhir) dan satu kali kembali ke HIGH, sehingga LED (1,1) hanya di-*toggle* satu kali. Pada pengujian yang sama, setiap LED berstatus ON mendapat anoda HIGH dan katoda LOW selama 33,3% waktu (satu dari tiga baris), sesuai rancangan multiplexing.



Gambar 24: Sinyal mentah kolom C1 dengan bouncing (atas) dibandingkan `stableState` hasil debouncing 50 ms (bawah) pada satu kali penekanan S11 (pengujian model, bukan Wokwi).

3.4 Jawaban Pertanyaan Analisis

1. **Mengapa debouncing tetap diperlukan walaupun hanya satu tombol yang ditekan pada satu waktu?**
   Bouncing tidak disebabkan oleh banyaknya tombol yang ditekan, tetapi oleh sifat mekanik setiap tombol itu sendiri. Seperti dijelaskan pada C.5, kontak logam di dalam tombol memantul beberapa kali saat bersentuhan dan saat terlepas, sehingga satu tombol saja sudah menghasilkan beberapa transisi HIGH–LOW dalam beberapa milidetik. Pada matrix, setiap tombol dibaca ulang setiap kali baris-nya di-scan, dan proses scan berlangsung jauh lebih cepat (orde mikrodetik) dibanding durasi getaran. Akibatnya getaran tersebut terbaca sebagai beberapa kali tekan. Karena program memakai mode *toggle*, satu penekanan yang terbaca dua kali akan membuat LED menyala lalu langsung padam lagi. Percobaan 4 membuktikan efek ini terjadi pada tombol yang sama: penghitung tanpa debounce selalu lebih besar. Oleh karena itu setiap tombol memerlukan debouncing sendiri, lengkap dengan variabel status dan waktunya masing-masing.

2. **Konfigurasi pull-up/pull-down pada kolom dan kondisi saat tombol tidak ditekan.**
   Rancangan ini memakai **pull-up internal** (`INPUT_PULLUP`) pada ketiga pin kolom (GPIO25, 26, 27). Saat tidak ada tombol yang ditekan, pin kolom terbaca **HIGH** karena ditarik ke 3,3 V oleh resistor internal. Ketika baris yang sedang di-scan ditarik LOW dan tombol di perpotongannya ditekan, kolom terhubung ke baris tersebut dan terbaca LOW. Kondisi ini tidak boleh dibiarkan *floating* karena pin input berimpedansi sangat tinggi akan mengambang ke nilai acak akibat derau, kopling dari jalur scan yang berganti-ganti, maupun sentuhan tangan. Jika kolom *floating*, program bisa membaca LOW padahal tidak ada tombol ditekan, sehingga LED berubah status dengan sendirinya. Debouncing pun tidak dapat menolong karena nilai acak tersebut bisa stabil lebih lama dari 50 ms. Selain itu, pin baris yang tidak di-scan dijaga HIGH (bukan dibiarkan mengambang) agar tombol pada baris lain tidak ikut menarik kolom menjadi LOW.

3. **Perbedaan membaca GPIO sebagai digital input dan analog input bila satu tombol diganti LDR.**
   Pada **digital input**, rangkaian pembanding (*Schmitt trigger*) di dalam pin hanya memutuskan

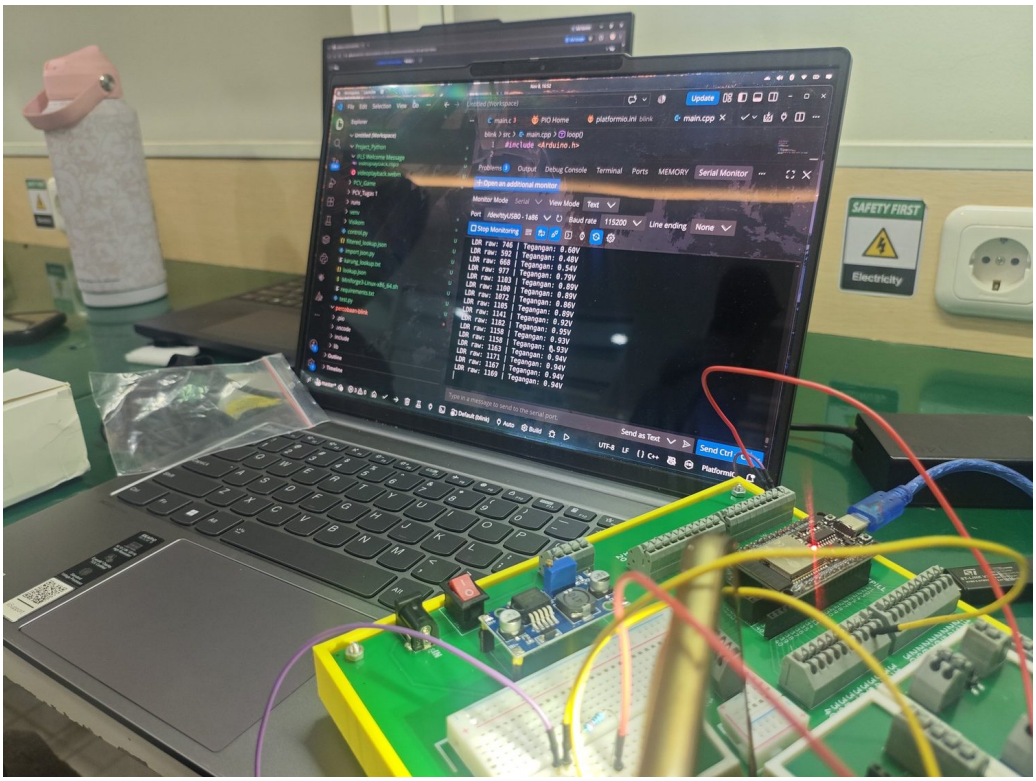
apakah tegangan berada di atas atau di bawah ambang logika, sehingga hasilnya hanya 1 bit: HIGH atau LOW. Semua tegangan di antara 0 dan 3,3 V dipaksa menjadi salah satu dari dua nilai tersebut. Pada **analog input**, tegangan diteruskan ke ADC SAR yang membandingkannya secara bertahap terhadap tegangan referensi sebanyak 12 kali (satu kali per bit). Hasilnya adalah bilangan 12 bit dengan $2^{12} = 4096$ tingkat, sehingga ADC dapat membedakan perubahan tegangan sekitar 0,8 mV. Inilah sebabnya Percobaan 5 menampilkan nilai seperti 0, 189, 1100, dan 2864 untuk kondisi gelap, ruangan, disenter, dan sangat terang. Pada matrix ini, LDR juga tidak dapat dipasang langsung pada perpotongan baris-kolom seperti tombol, karena LDR bukan saklar dan pin kolom yang dipakai (GPIO25–27) termasuk ADC2. LDR perlu dirangkai sebagai pembagi tegangan tersendiri pada pin ADC1 (misalnya GPIO34) dan nilainya diubah menjadi status tombol dengan ambang batas di dalam program, misalnya dianggap "ditekan" bila nilai raw di bawah 100.

4 Kesimpulan

1. PlatformIO pada VSCode berhasil digunakan sebagai satu lingkungan pengembangan untuk dua keluarga chip dan dua framework yang berbeda hanya dengan mengganti isi `platformio.ini`.
2. STM32 Blackpill berhasil di-*flash* melalui ST-Link dengan protokol SWD (OpenOCD, `Verified OK`), sedangkan ESP32 cukup diprogram lewat USB karena memiliki chip USB-to-Serial dan rangkaian auto-reset (`Hard resetting via RTS pin`). LED PC13 Blackpill bersifat aktif LOW, sedangkan LED GPIO2 ESP32 aktif HIGH.
3. Framework ESP-IDF memakai `app_main()`, FreeRTOS, dan driver GPIO yang eksplisit, sehingga lebih panjang namun lebih dekat ke perangkat keras dibanding Arduino yang memakai `setup()`/`loop()`.
4. Pull-up membuat pin bernilai HIGH saat tombol dilepas dan LOW saat ditekan, sedangkan pull-down sebaliknya. Resistor eksternal 10 kΩ dan resistor internal ESP32 memberikan hasil logika yang identik.
5. Bouncing terbukti terjadi pada tombol mekanik. Debouncing 50 ms memberikan hasil paling seimbang. Nilai 10 ms masih meloloskan getaran, sedangkan 200 ms membuang penekanan cepat yang sah (hanya 7 dari 36 transisi terhitung).
6. ADC 12-bit ESP32 berhasil membaca LDR sebagai besaran kontinu dari raw 0 (gelap) hingga sekitar 2864 (sangat terang), dengan resistansi LDR berubah dari ratusan kΩ hingga sekitar 4 kΩ. Pembacaan di dekat 0 V kurang linear sehingga pemilihan nilai resistor pembagi tegangan perlu disesuaikan dengan kondisi cahaya yang diukur.
7. Tugas Matrix Pushbutton 3x3 dan Matrix LED 3x3 dapat direalisasikan dengan 12 GPIO melalui teknik scanning dan multiplexing, dengan debouncing independen untuk setiap tombol.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 25: Dokumentasi saat praktikum Modul 1.